

Google Summer of Code 2026



Cuneiform Digital Library Initiative

Project Proposal: Search & Discovery Improvements

(Large Project, 350 hours)

Harsh Chandwani

Indian Institute of Technology Madras

BS (Bachelor of Science), 4th semester

Graduation Year: 2028



[harsh04044](#)



[harsh04044](#)



harsh9425181074@gmail.com



+91 9425181074

Indian Standard Time (+5:30 GMT)

Index

About Me	3
Technical Skills	3
Relevant Projects	3
Contributions to CDLI Framework	3
Project Proposal	5
Project Overview	5
The findings	6
How I Plan to Fix It	8
Testing Plan	14
Deliverables	15
What Changes After This Project	15
Implementation Timeline & Milestones	16
Interest & Motivation	17
Why GSoC	17
Why CDLI	17
Why This Project	17
Availability & Commitment	18
Post-GSoC	18

About Me

I am Harsh Chandwani, a 2nd year BS student at Indian Institute of Technology Madras. I started building web projects about a year and a half ago, mostly full-stack work with React, Node.js, and PHP. Over time I found myself more drawn to backend and systems work, which is what pulled me toward this project.

I got into open source through Hacktoberfest and have been contributing across different projects since then. Before writing this proposal I spent a few weeks going through the actual CDLI codebase and discussed the architecture with the maintainers on Slack. The specifics in this proposal come from that, not from reading the project description.

Technical Skills

- Backend: PHP (CakePHP 4), Node.js (Express), Python (Django REST Framework)
- Frontend: React, TypeScript, Next.js, Tailwind CSS
- Databases: PostgreSQL, MySQL/MariaDB, MongoDB
- Search: Elasticsearch: index mapping design, query DSL, incremental document updates. Directly relevant since CDLI uses the same REST API.
- ORMs: Prisma, Drizzle
- DevOps: Docker, Git/GitLab
- Open source: **48 merged pull requests** across different organizations working with PostgreSQL, Drizzle ORM, GraphQL, React, TypeScript, and Playwright.

Relevant Projects

- **Harvation:** visual workflow automation tool (n8n-style). Turborepo monorepo with Next.js, Express, BullMQ worker on Redis. Decouples job enqueueing from execution, same pattern as the Transactional Outbox in this project.
- **Full-Text Search Engine:** document indexing system on Elasticsearch. Wrote index mappings, query DSL layer, and incremental document updates. The document builder pattern maps directly to what ArtifactDocumentBuilder needs to do.
- **CakePHP REST API:** REST API for a CMS built on CakePHP 4. Worked with model associations, custom Commands, and model event listeners, the same patterns used throughout this project.

Contributions to CDLI Framework

All contributions are to the gitlab.com/cdli/framework repository.

MR	Description	Status
!1131	Added two sections to SEARCH.md documenting how the current nightly reindex works and which pages use Elasticsearch vs the database directly. Written while studying the codebase, it covers exactly the part of the system this project changes.	Merged
!1138	Removed a broken Uqnu editor link from the inscriptions edit page and kept the surrounding text as plain content.	Merged
!1139	Built an admin page that lists artifacts sharing the same museum number, designation, accession number, or excavation number. Includes pagination, a compare link for side-by-side review, and the SQL duplicate detection logic from issue #2354.	Merged
!1169	Wrapped all affected fields in h() to ensure consistent HTML escaping across both the full card and compact table views.	Merged
!1140	Added the first unit tests for the ElasticsearchQuery layer covering four helper methods: 7 tests, 20 assertions total. Established a pattern for exposing protected methods in tests. Part of issue #2525, directly preparatory for the OpenSearch migration.	Open
!1158	Implemented an auto-generated annotation flag on inscriptions and a helper that injects the marker into CDLI-CoNLL after the header block. The flag stays in sync via a beforeSave hook. Includes Elasticsearch indexing, CoNLL-U export support, and unit tests.	Open
!1172	Added structured logging for Elasticsearch search and count failures, covering transport errors, bad HTTP responses, and malformed JSON. Also logs slow searches when response time exceeds a configurable threshold. Fast searches stay silent to avoid log noise.	Open

All of my contributions to CDLI can be found at: [framework](#)

Most of my contributions touch the **search layer** directly, which is exactly what this project is about. Beyond CDLI, I have around **48 merged pull requests** in other open-source organizations, primarily working with PostgreSQL, Drizzle ORM, GraphQL APIs, React, TypeScript, and Playwright tests. A full list is available [here](#).

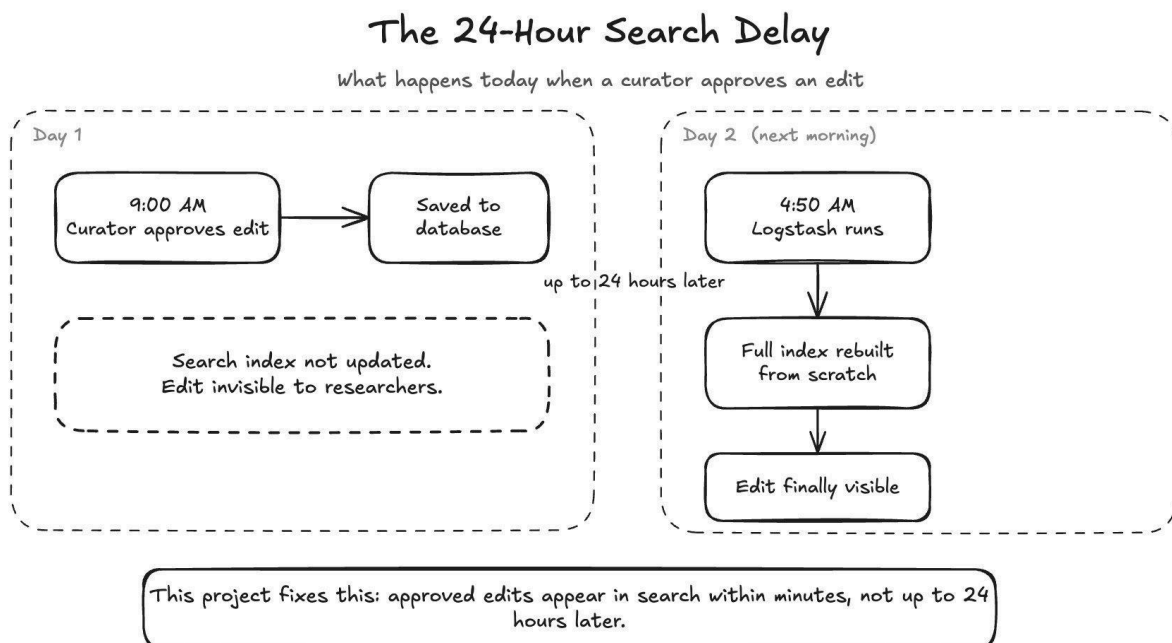
Project Proposal

Search & Discovery Improvements

Project Overview

The current setup uses Elasticsearch 7.17 inside a CakePHP backend. Indexing is handled by a Logstash pipeline that runs once a night at 04:50 AM, executing a 490-line SQL query with 18 JOINS to rebuild the entire index from scratch. I found three things while going through the codebase that need fixing:

- **No incremental indexing:** every run is a full reindex regardless of how many records changed. A curator who approves an edit at 9 AM will not see it in search results until the next morning.
- **Search is inconsistent across entity types:** artifacts go through Elasticsearch but publications, collections, proveniences, and periods all query MariaDB directly, so there is no way to do unified cross-entity search.
- **Licensing mismatch:** Elasticsearch's current licensing conflicts with how CDLI operates as an open-source project. OpenSearch is a community-maintained Apache 2.0 fork that preserves the same REST API, so switching is straightforward and keeps the stack fully open.



The plan is to replace the nightly rebuild with event-driven incremental indexing, where only the records affected by a change get reindexed within minutes of approval. Publications, collections,

proveniences, and periods will also move into OpenSearch so listing and search pages stop going to the database directly.

The findings

Before writing this proposal I spent time reading through the codebase. This section covers three things I found:

1. How the Current Indexing Works

The current indexing pipeline is defined in `logstash.conf` and runs once nightly via a JDBC connector. It executes a single 490-line SQL query against MariaDB with approximately 18 LEFT JOINS, including recursive CTEs for `artifact_type`, `material`, `genre`, and language hierarchies.

The query uses `GROUP_CONCAT` with three separator levels (`|`, `;`, and `:`) to pack related data into flat strings. A Ruby filter (`artifacts.rb`) then splits these strings and transforms them into the nested document structure that goes into the search index, parsing ATF transliteration line by line and normalizing diacritics via Unicode NFD decomposition.

Crucially, incremental tracking in `logstash.conf` is commented out. Every run is a full reindex. There is no event-driven mechanism anywhere in the codebase.

2. Where Incremental Indexing Plugs In

Incremental indexing means only reindexing the artifacts that actually changed, not the entire corpus. To make this work, we need a place in the existing code where we can trigger a reindex job the moment a change is approved. Reading through `approve()`, I noticed it already wraps everything in a transaction. After `commit()` there is a gap and nothing touches OpenSearch at all. That is exactly where the hook goes. By the time `commit()` runs, the affected artifact IDs are known, so a targeted reindex job can be queued right there.

```

// UpdateEventsController.php approve() BEFORE
if ($success) {
    $connection->commit();
    // gap: nothing happens in OpenSearch
    $this->Flash->success(__('The update has been approved.));
} else {
    $connection->rollback();
}

// UpdateEventsController.php approve() AFTER
if ($success) {
    $artifactIds = $event->getAffectedArtifactIds(); // new
    $this->ReindexOutbox->enqueue($artifactIds);      // new (inside transaction)
    $connection->commit();                          // commit all together
    $this->Flash->success(__('The update has been approved.));
} else {
    $connection->rollback();
}

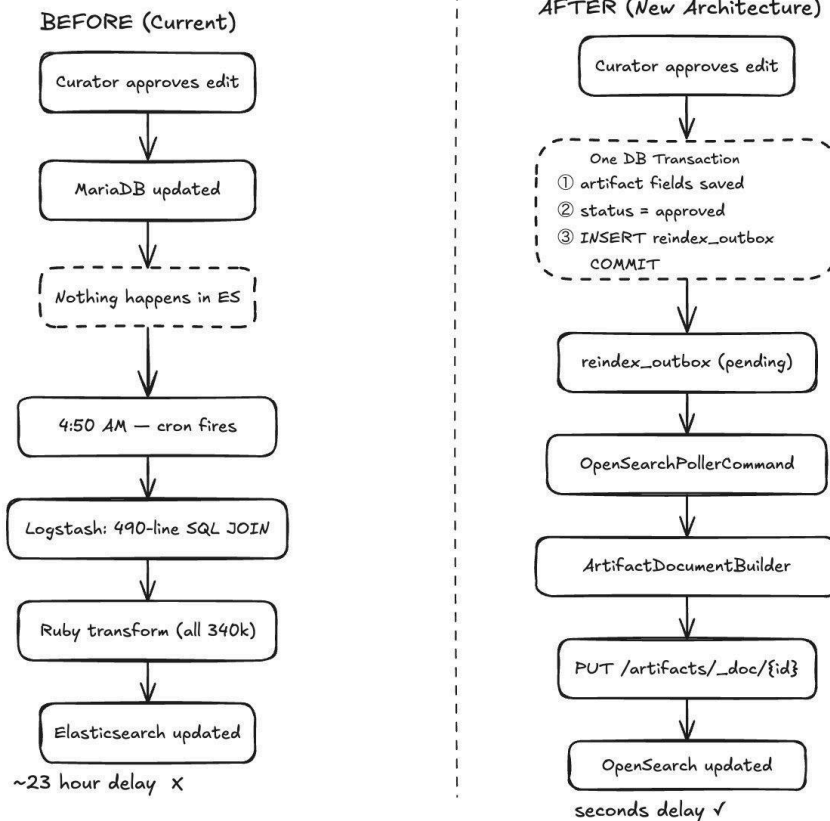
```

UpdateEventsController::approve() : current gap (BEFORE) vs proposed 2-line fix (AFTER)

3. Current Search Coverage by Entity Type

Entity	Current Search Backend	Problem
Artifacts / Inscriptions	Elasticsearch	Stale (nightly reindex only)
Publications	MariaDB direct query	No full-text search capability
Collections	MariaDB direct query	No full-text search capability
Periods	MariaDB direct query	No full-text search capability
Proveniences	MariaDB direct query	No full-text search capability

Before vs After



Current nightly Logstash pipeline vs event-driven OpenSearch architecture

How I Plan to Fix It

1. Switching to OpenSearch

Since ElasticSearchQuery.php uses raw HTTP POST with no SDK, the switch is primarily a configuration change:

- Swap the Elasticsearch container with OpenSearch in docker-compose.dev.yml
- Update the service URL in app.php
- OpenSearch maintains full compatibility with the Elasticsearch REST API, so ElasticSearchQuery.php continues to work without rewriting the query logic

The migration is also an opportunity to clean up a known mapping issue. Several free-text keyword subfields like artifact_comments and cdli_comments have ignore_above: 256, which silently drops values longer than 256 characters from term filters and aggregations. Since the migration requires

writing a new OpenSearch mapping from scratch, removing this limit for the affected fields is a natural part of the work.

2. Incremental Indexing via Transactional Outbox

How Transactional Outbox Solves the Indexing Problem

The Transactional Outbox is a pattern where instead of pushing a job to an external system, the reindex job is written as a row in a database table called **reindex_outbox**, inside the same transaction as the artifact change itself. A separate background process then reads from this table and pushes the updates to OpenSearch. This way the job is never lost even if something fails, because the database handles the reliability guarantee.

Since `approve()` already has a **BEGIN/COMMIT** wrapping everything, adding one **INSERT** to that transaction is trivial. If the transaction rolls back, the outbox row goes with it. If the poller crashes halfway through, the row stays pending and gets retried. Failures become visible and recoverable instead of silent and permanent.

How it works:

- When an update is approved, the existing transaction is extended to also **INSERT** a row into a new **reindex_outbox** table with the affected artifact IDs and a status of pending. If the transaction rolls back, the outbox row disappears with it, so there are no ghost jobs.
- A background poller command runs on a short cron interval. It picks up pending rows from the outbox, builds the OpenSearch documents, pushes the updates, and marks jobs as done. Failed jobs are retried with exponential backoff.
- If the poller picks up the same job twice by accident, it does not matter. **PUT** to OpenSearch just overwrites the existing document with the same data, so nothing breaks.

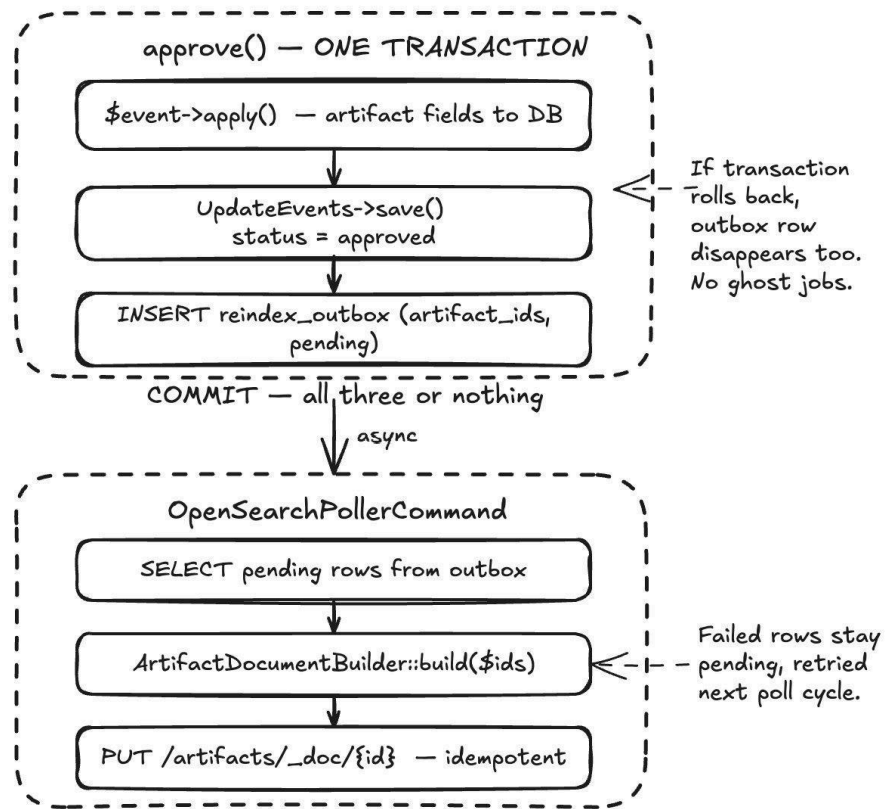
Why We Are Not Using Redis

The CDLI stack already has Redis running for image processing jobs. My first thought was to reuse it for indexing too, and when I discussed this with the maintainers on Slack they said they would be open to it.

See the Slack discussion: digcuneiform.slack.com/archives/C9V81L894/p1772299017636419

But Redis creates a reliability problem: `approve()` commits to the database, then tries to push a job to Redis. If that Redis write fails, the job is gone forever. The database has the change but OpenSearch never finds out. For a research database that means a curator's approved edit is silently invisible to researchers with no automatic way to catch it. The Transactional Outbox avoids this entirely since the job lives in the same database transaction as the change.

Transactional Outbox Flow



Transactional Outbox flow: `approve()` transaction and background poller

Outbox Table Schema

Column	Type	Description
id	BIGINT AUTO_INCREMENT	Primary key
artifact_ids	JSON	Array of artifact IDs to reindex
event_type	VARCHAR(50)	artifact_update, inscription_change, cascade_collection, etc.
status	ENUM	pending, processing, done, failed
retry_count	TINYINT	Number of retry attempts (max 3)
created_at	DATETIME	Timestamp of job creation
processed_at	DATETIME	Timestamp of successful completion
error	TEXT	Last error message if failed

```
CREATE TABLE reindex_outbox (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  artifact_ids JSON          NOT NULL,
  event_type  VARCHAR(50)    NOT NULL,
  status      ENUM('pending','processing','done','failed')
              DEFAULT 'pending',
  retry_count TINYINT        DEFAULT 0,
  created_at  DATETIME        DEFAULT NOW(),
  processed_at DATETIME       NULL,
  error       TEXT           NULL,
  INDEX idx_pending (status, created_at)
);
```

reindex_outbox table schema: the INDEX on (status, created_at) ensures the poller query is fast

3. PHP Document Builder

Currently Logstash uses a Ruby script to take raw database records and convert them into the structured format that OpenSearch needs. This project replaces that Ruby script with a PHP class that does the same job but only for the specific artifacts that changed, not the entire corpus. This class is the core of the incremental indexing system.

Looking at artifacts.rb, the PHP builder needs to handle seven things the Ruby filter currently does:

```
class ArtifactDocumentBuilder
{
  public function build(array $artifactIds): array {}
  private function fetchData(array $ids): array {}
  private function removeDiacritics(string $text): string {}
  private function graftProvenienceAr(
    string $english,
    ?string $arabic
  ): string {}
  private function consolidateIds(array $data): array {}
  private function parseAtf(string $raw): array {}
  private function zipPublications(array $data): array {}
  private function zipAssets(array $data): array {}
  private function zipExternalResources(array $data): array {}
  private function zipUpdateEvents(array $data): array {}
}
```

PHP Document Builder class skeleton: each method maps to one stage in artifacts.rb

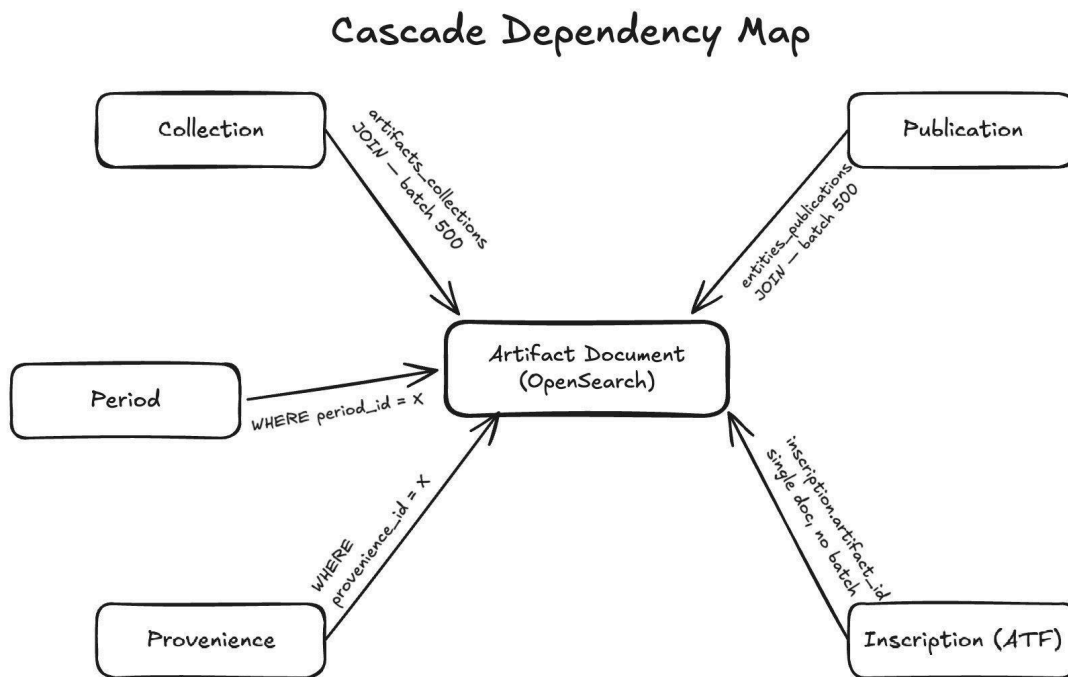
- **Diacritics normalization:** normalizes special characters in text fields to produce plain ASCII versions for search. The Arabic translation field needs special handling since it has to be merged into the English string in a specific format.
- **ID consolidation:** builds the `cdli_id` array from the P-number, composite number, and seal number. Also pulls in all related composite and seal numbers for the artifact.
- **Parallel array zipping:** the current pipeline packs related data into flat strings using separator characters. The PHP builder instead fetches proper arrays directly from the database, which is cleaner and easier to maintain.
- **Timestamps:** the document's created and modified dates come from min and max of `update_events.approved` across all approved events for that artifact, not from `artifacts.created/updated`.
- **Publication references:** ports the reference formatting logic from the Ruby filter. If a reference starts with punctuation it gets joined directly to the designation, otherwise a comma and space go in between.
- **ATF parser:** a stateful parser that reads raw ATF text line by line and dispatches on line prefixes to build `atf_transliteration` and related fields. One subtle behaviour that must be replicated exactly: if a surface block has no text yet, the parser reuses the last surface entry instead of creating a new one.
- **Recursive CTEs:** the SQL uses recursive CTEs to build full hierarchy paths like Tool > Knife. CakePHP ORM cannot do recursive CTEs so these will run as raw SQL with the artifact IDs passed as parameters.

4. Keeping Related Records in Sync

OpenSearch stores a complete copy of each artifact's data in one place, including the period name, provenience, and collection it belongs to. This makes search fast, but it means any change to shared data like a period name or collection needs to be reflected across all artifacts that reference it. Here is what needs to be tracked:

Entity Changed	Artifacts Affected	Fields Impacted
Collection name/license	All artifacts in that collection	Collection [], collection_ar [], collection_ascii [], license fields
Period name/sequence	All artifacts for that period	period, period_ar, period_sequence, period_comments

Entity Changed	Artifacts Affected	Fields Impacted
Provenience location	All artifacts from that site	provenience, provenience_ar, region, written_in, findspot_*
Publication metadata	All artifacts citing that publication	Full publication [] nested array
Inscription ATF/JTF	Single artifact via inscription.artifact_id	All 7 atf_* fields, requires full ATF re-parse



afterSaveCommit() hooks on each entity table trigger the cascade

Cascade dependency map: which entity changes trigger artifact document reindexing

CakePHP Model afterSaveCommit() hooks on CollectionsTable, PeriodsTable, and ProveniencesTable will detect these changes and batch the affected artifact IDs into outbox rows. For large cascades (a period change could affect tens of thousands of artifacts), IDs will be batched in groups of 500.

5. Making Other Entities Searchable

Publications, collections, proveniences, and periods currently have no search infrastructure at all. Every search against these entities hits MariaDB directly. This project will:

- Design flat OpenSearch indexes for each entity with document builders tailored to their simpler data models

- Refactor the corresponding controllers to query OpenSearch instead of the database for listing and search pages
- Add `afterSaveCommit()` hooks for each entity to keep its index current when data changes

6. Full Reindex Command

A `bin/cake Command` will handle the initial migration from Elasticsearch to OpenSearch and act as a recovery tool if the index ever gets out of sync. It processes artifacts in batches of 500 to avoid memory issues on the full 340k+ corpus:

```
bin/cake opensearch reindex --entity=artifacts --batch=500
bin/cake opensearch reindex --entity=publications
bin/cake opensearch reindex --entity=all
```

The codebase already has an admin page with an AJAX progress bar for index operations. The full reindex will follow the same pattern so admins can trigger it and watch the progress.

Testing Plan

Here is how I plan to test each major component:

- **PHP Document Builder:** run the existing Logstash pipeline on a set of known artifacts, capture the output, then run the PHP builder on the same artifacts and compare field by field. Any difference is a bug. The ATF parser gets its own test file covering every dispatch branch and the reuse-last-surface-if-empty edge case.
- **Transactional Outbox:** write an integration test that calls `approve()` on a test artifact, verifies the outbox row was inserted, runs the poller manually, and checks the OpenSearch document was updated correctly. Also test the failure path: mark a job as failed mid-way and verify it stays pending and gets retried.
- **ElasticSearchQuery layer:** there are currently no tests for this. The plan is to start with helper methods like `transformFieldName()` and `makeBoolQuery()` to establish the pattern, then expand to the core query construction pipeline. After the OpenSearch swap, run all tests and fix whatever breaks.
- **Cascade hooks:** change a Collection name, Period name, and Provenience and verify the correct artifact IDs end up in the outbox. Also test batch size limiting by simulating a period that affects more than 500 artifacts and checking that multiple outbox rows are created.

Deliverables

- OpenSearch index mappings for all major searchable entities, with the silent truncation bug fixed in free-text fields and wildcard field types preserved for ATF transliteration search.
- Existing Elasticsearch query builders, result set parsers, and paginators migrated to OpenSearch, with transliteration regex, nested aggregations, keyset pagination, and access control all still working.
- Transactional Outbox table and CakePHP poller Command that reacts to database changes and updates only the impacted OpenSearch documents, with safe retries.
- PHP Document Builder class replicating all seven transformation stages of the Logstash Ruby filter, with unit tests covering ATF parser edge cases, diacritics normalization, and parallel array zipping.
- Cascade hooks on Collection, Period, and Provenience tables that detect shared metadata changes and batch-enqueue reindex jobs for all affected artifacts.
- OpenSearch indexes and document builders for Publications, Collections, Proveniences, and Periods with the corresponding controllers refactored to use OpenSearch for listing pages.
- Full reindex bin/cake Command and admin progress UI.
- Developer documentation covering OpenSearch setup, index design, the Transactional Outbox approach, and how to run the full reindex locally.

What Changes After This Project

- Curators will see their approved edits appear in search results within minutes, not the next morning.
- Search and listing pages stop hitting the database directly, reducing load on MariaDB.
- Publications, collections, proveniences, and periods become searchable through the same system as artifacts.
- The search stack moves to a fully open-source license, which is one of the main reasons this migration is happening.
- The search layer gains meaningful test coverage for the core query construction methods that currently have none.
- Documentation good enough that a new contributor can work on search without needing to read the entire DB schema first.

Implementation Timeline & Milestones

Phase	Milestones	Dates
Community Bonding	Meet with mentors and finalize scope. Get the local OpenSearch environment running and verify it works with the existing codebase. Discuss the test strategy and agree on the overall approach before writing any code.	May 1 – May 24
Week 1 – 2	Swap Elasticsearch with OpenSearch in the local dev environment and verify the existing search still works. Check for any API differences that need fixing. Write the first unit tests for the search query layer and run them against OpenSearch.	May 25 – June 7
Week 3	Design the new OpenSearch index mappings for artifacts and all other entity types. Fix the silent truncation issue in free-text fields. Make sure the fields needed for transliteration search are correctly configured.	June 8 – June 14
Week 4 – 5	Build the core of the PHP document builder that replaces the Ruby transform. Handle data fetching, text normalization, ID consolidation, and all the nested object transforms. Write tests for each part and verify the output is correct.	June 15 – June 28
Week 6	Port the ATF transliteration parser from Ruby to PHP. This is the most complex part of the document builder. Write tests for all the parsing cases and compare the output against what the current pipeline produces.	June 29 – July 5
Week 7 + Midterm	Finish the document builder with publication and timestamp logic. Write a full comparison test against the current Logstash output. By the midterm deadline (July 10), OpenSearch should be running with a complete document builder and a working initial reindex.	July 6 – July 12
Week 8	Create the outbox table that stores reindex jobs. Hook the artifact approval flow into it so every approved edit automatically gets queued. The outbox insert happens inside the same database transaction as the approval, so nothing gets lost if something fails.	July 13 – July 19
Week 9	Build the background poller that picks up jobs from the outbox and sends them to OpenSearch. Add retry logic for failed jobs. Write integration tests that cover the full flow from approval through to the index being updated.	July 20 – July 26
Week 10	Add hooks so changes to shared entities like collections, periods, and proveniences automatically trigger reindex jobs for all affected artifacts. Test that the right artifacts get queued when a shared field changes.	July 27 – August 2

Phase	Milestones	Dates
Week 11	Build OpenSearch indexes for publications, collections, proveniences, and periods. Update the listing and search pages for these entities to read from OpenSearch instead of the database.	August 3 – August 9
Week 12	Add a reindex command for running a full index rebuild when needed, such as during the initial migration or if the index gets out of sync. Remove Logstash from docker-compose. Build an admin page with a progress bar.	August 10 – August 16
Final Week	Write developer documentation covering setup, index design, and the outbox pattern. Write the final GSoC report and a blog post. Address any outstanding review feedback before the August 24 deadline.	August 17 – August 24

Interest & Motivation

Why GSoC

I started contributing to open source about six months ago. GSoC is a chance to take that further: instead of individual PRs, owning a complete feature from design to documentation to production. That kind of end-to-end experience is harder to get through regular contributions alone.

Why CDLI

I came across CDLI while looking at GSoC orgs and spent a few hours just reading about what the project actually does. Hosting 340k cuneiform tablets and making them searchable to researchers worldwide is a genuinely interesting problem. Most open-source projects I looked at were either too small to be meaningful or too large for a single contributor to have real impact. CDLI felt like the right size: a real production system with real users, where a focused project can make a noticeable difference.

The maintainers are also active on Slack and willing to discuss architecture before you write code. That made a difference when I was working through the design decisions in this proposal.

Why This Project

The Search and Discovery project stood out because the problem is specific enough to be solvable in one summer but deep enough that you have to understand the whole system to solve it. Fixing the nightly Logstash rebuild means touching the indexing pipeline, the query engine, the approval workflow, and the model relationships. I find that kind of problem more interesting than something contained in a single file or layer.

The other reason is that **I had done relevant work before**: a job queue system, a document builder for Elasticsearch, a CakePHP API. When I read the project description and then the actual codebase, I recognised the same patterns I had worked with before. That gave me enough confidence to go deep into the codebase before writing the proposal, which is how I ended up reading `artifacts.rb` line by line and mapping out the cascade dependency graph.

Availability & Commitment

My college has a 0% attendance requirement, so I can work full time on this regardless of whether the semester is running. During the coding period I can put in 35-40 hours per week consistently. I am in IST (UTC +5:30) and can adjust my schedule for calls or reviews with mentors in other timezones.

I have no exams or travel planned during the coding period. If something comes up I will flag it early.

The Slack conversations I have already had with the maintainers show how I prefer to work: share what I am thinking before committing to it, ask specific questions, update the design based on feedback. I plan to keep working that way throughout the project.

I am 100% dedicated to [CDLI](#) and have nothing else planned this summer.

Post-GSoC

After GSoC ends I plan to stay involved in a few ways.

The Transactional Outbox poller and the PHP Document Builder will probably need adjustments once they hit real usage. I will keep an eye on issues related to these and fix bugs as they come up.

The search layer has very little test coverage right now. My MR [!1140](#) makes a start but the core query construction methods are still untested. By the end of GSoC the main paths should be covered, but there will still be more to add. I plan to keep contributing tests after the program ends.

Additionally, I would love to maintain the code I wrote during GSoC and continue learning from epp and lars and other mentors and contributors, and hopefully mentor for GSoC 2027.